

University of Zagreb
Faculty of Science
Department of Mathematics

Duje Perković, Matko Petričić, Paula Horvat

Convolutional Neural Networks

Zagreb, May 2025.

Contents

1	Definitions and Ideas	2
1.1	Introduction	2
1.2	Assumptions on the Data (Experience E)	2
1.3	How to Measure Error? (Accuracy P)	3
1.4	Selection of Function Family	4
1.5	Defining a Neural Network	5
2	Convolutional Neural Networks	7
2.1	Introduction and Motivation	7
2.2	Main Components of a Convolutional Neural Network	7
2.2.1	Convolutional Layer	8
2.2.2	Activation Function	9
2.2.3	Pooling Layer	11
2.2.4	Output Layer	11
3	Forming the Algorithm	13
3.1	How to Compute the Optimal Coefficients θ	13
3.1.1	Gradient Descent	13
3.1.2	Stochastic Gradient Descent	14
3.1.3	Momentum	15
3.2	Computing Gradients	16
3.3	Initializing the Recursion	17
3.3.1	Top K Loss Function	18
3.3.2	Softmax	18
3.3.3	Log Loss Function	18
3.3.4	Softmax Log Loss Function	18
3.3.5	p-Norms, $1 \leq p < \infty$	19
3.4	Carrying Out the Recursion – Activation Functions	19
3.4.1	ReLU	19
3.4.2	Sigmoid – Logistic Function	20
3.5	Carrying Out the Recursion – Pooling Layers	21
3.5.1	Max Pooling	22

4	Experimental Results	25
4.1	Model 1 – Basic CNN Architecture	25
4.2	Model 2 – Extended CNN Architecture with Additional Layers	28
4.3	Recommendations for Model 3	30
4.4	Model 3 – Optimized CNN with BatchNorm and Stronger Regularization . .	30
4.5	Conclusion of the Experimental Part	32
5	Conclusion	33
6	References	34

Abstract

This paper examines *convolutional neural networks* (CNNs), one of the most important and successful deep learning architectures used in image processing and computer vision. CNNs are particularly effective because they exploit the spatial structure of input data, enabling the recognition of patterns such as edges, shapes, and objects within an image. Classical neural networks are not sufficiently efficient for high-dimensional data such as images, since they ignore local connectivity and spatial structure. Convolutional networks address these limitations by introducing components such as *convolutional* and *pooling* layers, which allow features to be extracted without losing spatial information.

The main building block of a CNN is the *convolutional layer*, which uses filters to compute local patterns within an image. It is typically combined with activation functions (most commonly ReLU), dimensionality-reduction layers (e.g., max pooling), *flatten* layers, and fully connected layers for classification. Learning is carried out using backpropagation and optimization algorithms such as gradient descent.

Convolutional networks are successfully applied in object and face recognition, medical diagnostics, autonomous vehicles, satellite image analysis, and many other domains. Their effectiveness stems from their ability to learn patterns automatically and to process data hierarchically.

1 Definitions and Ideas

1.1 Introduction

Before introducing the formal definition of a neural network, we first present the motivation and intuition behind its construction.

Let a dataset $\mathcal{X} \subset \mathbb{R}^N$ and a finite set of labels $\mathcal{Y}$ be given, so that each data point has **exactly one associated label**; that is, there exists an assignment $y : \mathcal{X} \rightarrow S$.

The objective is to find an approximation of this function using some finite subset $\mathcal{V} \subset \mathcal{X}$ for which the labels are known. Intuitively, we expect that a larger subset will allow us to obtain a better approximation. The procedure of finding this function on the basis of this principle is called **machine learning**.

There is no single strict formal definition, but the underlying idea is the following: if we have some experience E from which an algorithm “learns”, a task T , and a performance measure P for solving that task, then increasing the amount of experience E should increase the value of the measure P .

It is easy to imagine various candidates for the task T : detecting tumors in CT or MRI images, determining the species of a plant from a photograph, or recognizing a person’s emotional state from facial expression. In the following paragraphs, we discuss the remaining concepts in more detail.

1.2 Assumptions on the Data (Experience E)

In general, data are divided into a **training** set and a **test** set. The first set is used in the learning procedure itself, while the second is used to determine the accuracy P .

Various statistical tools used to analyze the convergence of machine learning algorithms often require the following natural assumption:

- The dataset $\mathcal{X}$ is described by some distribution, and the finite subset $\mathcal{V} \subset \mathcal{X}$ is an independent sample.

This assumption makes it possible to establish a relationship between training error and test error. It motivates the interpretation of learning as **reducing the training error** in order to **reduce the test error**. It is worth noting that, in many cases, an additional intermediate set is also used: the **validation set**. It is used for fine-tuning hyperparameters and monitoring model generalization during training, before the final evaluation on the test set. In practice, a

common data split is 80% for training and 20% for testing, or 80% – 10% – 10% when the data are further divided into training, validation, and test sets.

1.3 How to Measure Error? (Accuracy P)

First, it is necessary to introduce the **loss function** $\mathcal{L}$, which compares the classification of a data point x_i produced by the approximate solution $\hat{\psi}$ with the exact solution ψ ; we require this function to be non-negative.

The simplest example is the **0-1** function given by:

$$\mathcal{L}_{0-1}(\hat{\psi}_i, \psi_i) = \begin{cases} 0 & \text{if } \hat{\psi}_i = \psi_i \\ 1 & \text{otherwise } \hat{\psi}_i \neq \psi_i \end{cases}$$

In practice, more complex functions are used depending on the type of problem, since we often want a more detailed measure of the deviation of the approximate solution. For example, in classification problems with C classes, the output is a column vector in $\mathbb{R}^C$ whose components are non-negative and sum to 1. Motivated by information theory, we use **cross-entropy**:

$$\mathcal{L}_{\text{CCE}}(\psi_i, \hat{\psi}_i) = - \sum_{j=1}^C \psi_{i,j} \log(\hat{\psi}_{i,j}) = -\log(\hat{\psi}_{i,c})$$

where $c \in 1, \dots, C$ is the class to which the element x_i belongs. In regression problems, squared and absolute loss functions are used:

$$\mathcal{L}_2(\psi_i, \hat{\psi}_i) = \|\psi_i - \hat{\psi}_i\|_2^2$$

$$\mathcal{L}_1(\psi_i, \hat{\psi}_i) = \|\psi_i - \hat{\psi}_i\|_1$$

Once the loss function has been chosen, the goal is to minimize the loss over all data in the training set. Using p_{data} to denote the distribution of the entire dataset, x for the input data, f for the classification function, y for the target output, and $f(x)$ for the obtained output, we write:

$$J^*(f) = \mathbb{E}_{x,y \sim p_{\text{data}}} L(f(x), y).$$

This quantity, minimized over a chosen family of functions, is called the **risk**.

The main difficulty is, of course, that the distribution of the entire dataset is unknown. Motivated by the Glivenko-Cantelli theorem, we therefore resort to the **empirical distribution**:

we use a finite training set and its distribution $\hat{p}_{data}$. Thus, we minimize the **empirical risk**:

$$\hat{J}^*(f) = \mathbb{E}_{x,y \sim \hat{p}_{data}} L(f(x), y).$$

For example, if we take the squared loss function and let m be the number of samples in the training set, then the problem becomes the minimization of the expression:

$$\hat{J}^*(f) = \frac{1}{m} \sum_{i=1}^m \|f(x_i) - y_i\|_2^2$$

Overall, the important distinction between ordinary optimization and machine learning is that the functional J^* , which we would ideally minimize but cannot in practice because the distribution p_{data} is unknown, is **replaced** by the problem of minimizing the empirical functional $\hat{J}^*$. We hope that this replacement is sufficient; more advanced theory confirms this intuitive idea under additional assumptions.

1.4 Selection of Function Family

From the preceding discussion, we see that machine learning requires the minimization of a functional, which is itself a complex theoretical and numerical problem.

We therefore restrict ourselves to a family of functions that can be described conveniently by a finite set of parameters θ . This allows us to use optimization algorithms such as gradient descent, which will form the basis for constructing a **neural network**.

In particular, the chosen family of functions must be rich enough to avoid **underfitting**, but not so unconstrained that it leads to **overfitting**.

Underfitting is the inability to minimize empirical risk. A standard example is approximating the XOR function with a linear model, i.e., a line (so that the set θ has two elements).

Formally, we try to approximate a binary classifier with a linear model, which reduces to the problem of dividing the plane so that one half-plane contains one class and the other half-plane contains the other. Recall that XOR is defined on the domain $\{(0,0), (1,0), (0,1), (1,1)\}$ and returns 1 if the argument contains exactly one unit, and 0 otherwise. The figure illustrates the impossibility of separating the plane in this way. The main purpose is not to provide a formal proof, but to motivate the use of more complex and richer families of functions for solving machine learning problems.

Overfitting occurs much more frequently in practice. It means that we have achieved a good minimization of empirical risk, but not of the true risk.

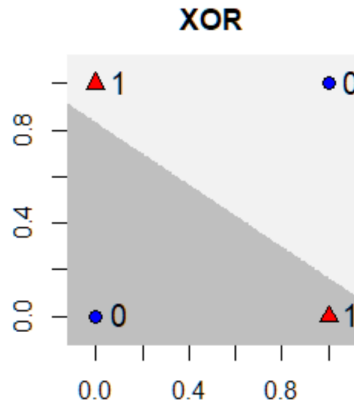


Figure 1.1: Impossibility of separating the plane

1.5 Defining a Neural Network

We now formally define the family of functions over which the minimization problem is solved:

Definition 1 (Neural Network). *Let a family of **layers** $\mathcal{L}$ be given.*

*The family of layers $\mathcal{L}$ contains, in particular, **activation layers** and layers of **feature functions**.*

*Then a **neural network** of depth $k \in \mathbb{N}$ is a function f obtained as the composition of k layers:*

$$f = L_k \circ L_{k-1} \circ \dots \circ L_1$$

Of course, such a definition is almost useless on its own, because one must still define activation layers, feature-function layers, and other possible layers that are used in practice and whose effects on the neural network are understood.

It is worth noting that there are more complex examples of neural networks defined by **directed acyclic graphs** (DAGs). For the purposes of this seminar, however, we focus on the simplest case: a one-directional chain, i.e., ordinary composition.

Activation functions are easier to explain: they are mappings whose main role is to introduce **nonlinearity** into the neural network, thereby helping to avoid underfitting. Common examples include the logistic function, tanh, and ReLU.

In order to satisfy the condition that neural networks are described by a finite set of parameters θ , we require that the **feature functions** and additional layers in $\mathcal{L}$ be parametrizable; that is, we identify a finite set of variables θ that parametrizes them.

In the following chapters, we present concrete examples of layers used in the construction of

neural networks, especially convolutional neural networks.

2 Convolutional Neural Networks

2.1 Introduction and Motivation

A convolutional neural network (CNN) is a special type of neural network designed for working with data that has a grid structure, such as images (2D grid of pixels) or time series (1D grid over time). Unlike ordinary neural networks that treat input data as vectors ($\in \mathbb{R}^{n \times 1}$), CNNs preserve the spatial structure of the data. Therefore, they are most commonly used and are very effective at recognizing patterns, shapes, textures, or objects in images.

Why and when should convolutional neural networks be used?

If we were to use a regular neural network for an image of size 100×100 pixels:

- This would give 10,000 inputs in only one layer.
- Even a small hidden layer would require millions of weights, which is inefficient and prone to overfitting.

Convolutional neural networks solve this problem using two key ideas:

1. **Local connections** – the network examines relationships among small parts of the image rather than all pixels at once; that is, a neuron in a CNN is not connected to the entire image, but only to a local region. This is effective because local properties, such as edges, lines, and curvatures, contain basic information about the image.
2. **Weight sharing** – the same pattern detector is used across the entire image; that is, we use a small filter (weight matrix) that moves across the image and is applied repeatedly to every part of it. In this way, far fewer parameters are required than in an ordinary neural network.

These ideas are inspired by the human visual system: we examine local parts of a scene and use the same “filters” in different parts of the image.

2.2 Main Components of a Convolutional Neural Network

Every CNN consists of several key components:

1. Convolutional layer

- Applies small filters (e.g., 3×3 or 5×5) across the image.

- Each filter learns to recognize a specific pattern, such as edges or textures.
- The result is a feature map.

2. Activation function

- Most commonly ReLU.
- Introduces nonlinearity into the network, enabling it to learn complex patterns.
- ReLU is often used because it mitigates the vanishing-gradient problem and accelerates learning.

3. Pooling layer

- Reduces the dimensions of the image.
- Helps the network become robust to small shifts of objects in the image.

We now discuss each component of the network in detail.

2.2.1 Convolutional Layer

As already mentioned, the role of a convolutional layer is to apply a set of filters, or kernels, to the input (for example, an image) in order to produce feature maps that emphasize patterns in the input, such as edges or particular shapes.

Figure 2.1 shows a simple example illustrating how a filter, i.e., a kernel, operates. Suppose that the initial image is represented as a matrix of size 6×6 . A smaller kernel, in this case of size 3×3 , is trained to recognize a particular feature, such as a straight line or a circle. The kernel acts on the original image by multiplying each 3×3 submatrix of the original matrix elementwise. Thus, the example in the figure shows how a 3×3 part of the original image is isolated and multiplied by the weight matrix, i.e., the kernel, yielding the result $429 = 18 \cdot 1 + 51 \cdot 1 + 121 \cdot 1 + 35 \cdot 1 + 204 \cdot 1$. The multiplication then continues analogously over all 3×3 submatrices to obtain the final result: a matrix of size 4×4 that provides relevant information about the image. This matrix is called a feature map. Figure 2.2 presents a complete example of convolutional multiplication.

INPUT IMAGE						WEIGHT			
18	54	51	239	244	188	1	0	1	429
55	121	75	78	95	88	0	1	0	
35	24	204	113	109	221	1	0	1	
3	154	104	235	25	130				
15	253	225	159	78	233				
68	85	180	214	245	0				

Figure 2.1

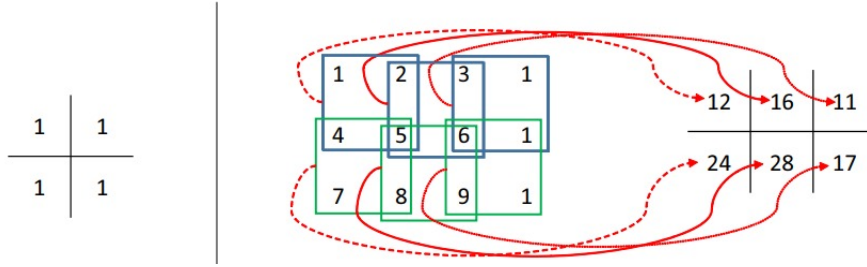


Figure 2.2

The weight matrix behaves like a filter on the image, extracting specific information from the original image matrix. One combination of weights may extract edges, another may identify a particular color, while a third may blur unwanted noise. The weights are learned by minimizing an error function and extracting patterns from the original image that help the network make accurate predictions. When multiple convolutional layers are used, the initial layers extract more general characteristics, while deeper layers learn increasingly complex features.

For color images, which are represented as third-order tensors, the convolution operation is defined in a similar way. Suppose that the input is a third-order tensor of size $H \times W \times D$. The convolutional kernel is also a third-order tensor of size $H' \times W' \times D$. When the kernel is placed on the input tensor at the spatial location $(0, 0, 0)$, we compute the products of the corresponding elements across all D components and sum the $H' \cdot W' \cdot D$ products to obtain the convolution result at that spatial location. The kernel is then moved from top to bottom and from left to right until the convolution is completed.

2.2.2 Activation Function

An activation function is a mathematical function applied after a linear operation, such as convolution or matrix multiplication. In a CNN, a convolutional layer computes a weighted sum of pixel values in a region of an image using a filter, which is simply a linear combination.

Without nonlinearity, the entire convolutional network, regardless of the number of layers, would be equivalent to a single linear transformation. This means that it would have difficulty learning more complex image characteristics. Real features, such as the shape of a cat, a nose, or a tree, are not linear. Therefore, the network must be able to model nonlinear functions. The role of the activation function is thus to enable the network to combine simple linear filters into a system capable of approximating complex functions.

The ReLU activation function is defined by:

$$\text{ReLU}(x) = \max\{0, x\}$$

Other nonlinear transformations are used in neural networks to introduce nonlinearity, for example the sigmoid function:

$$y = \sigma(x) = \frac{1}{1 + \exp(-x)}.$$

However, this function performs significantly worse than ReLU when training CNNs. If the sigmoid function is used, then $0 < y < 1$, and

$$\frac{dy}{dx} = y(1 - y).$$

Therefore,

$$\frac{dy}{dx} \leq \frac{1}{4}.$$

Thus, during error backpropagation, the gradient

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \cdot \frac{dy}{dx}$$

will have a substantially smaller absolute value than $\frac{\partial z}{\partial y}$ (at most $\frac{1}{4}$). The sigmoid function therefore causes the gradient to shrink considerably, and after several such layers the gradient will vanish, meaning that all of its components will be very close to zero. Vanishing gradients make gradient-based learning very difficult.

For the ReLU function, the situation is different. Consider the case where the input x is a third-order tensor.

$$\frac{dy_{i,j,d}}{dx_{i,j,d}} = [x_{i,j,d} > 0],$$

where $[\cdot]$ denotes the indicator function, which is 1 if the condition is satisfied and 0 other-

wise.

Accordingly, we have:

$$\left[\frac{\partial z}{\partial x} \right]_{i,j,d} = \begin{cases} \left[\frac{\partial z}{\partial y} \right]_{i,j,d}, & \text{if } x_{i,j,d} > 0 \\ 0, & \text{otherwise.} \end{cases}$$

Thus, the ReLU layer sets the gradients of some features to zero, meaning that those features are not activated (or are not relevant). For the features that are activated, the gradient is propagated backward without any change, which is useful for learning with SGD (Stochastic Gradient Descent).

2.2.3 Pooling Layer

If images have large dimensions, the number of trainable parameters must be reduced. For this reason, pooling layers are used between convolutional layers. Pooling reduces the spatial dimensions of the image and is applied independently to each depth component, i.e., to each frontal slice, so the depth of the image remains unchanged. Max pooling is the most commonly used type of pooling layer and is shown in Figure 2.3.

429	505	686	856
261	792	412	640
633	653	851	751
608	913	713	657

792	856
913	851

Figure 2.3

2.2.4 Output Layer

After the data pass through all convolutional layers and possible pooling operations, the end of the network must produce an output in the form of a class, i.e., a decision about which class the input belongs to. To generate the final output, a fully connected layer is applied, with output dimension equal to the number of classes we wish to recognize. This layer is the same as the final layer of an ordinary neural network.

Convolutional layers produce maps of various dimensions, while in the final step we are interested in one concrete outcome: whether the image belongs to a particular class or not. The

output layer uses a loss function, most commonly categorical cross-entropy, to compute the prediction error. After the forward pass is completed, the network performs backpropagation, which updates the neural network parameters in order to reduce the error and the loss.

Summary:

- The input image is passed to the first convolutional layer. The convolved output is obtained as an activation map. The filters applied in this layer extract relevant features from the image so that they can be propagated further through the network.
- Each filter detects a different characteristic that helps achieve accurate classification.
- Pooling layers are then added to further reduce the number of parameters in the network.
- Multiple convolutional and pooling layers are added before the prediction phase. Convolutional layers help extract features. The deeper we go into the network, the more specific the features become, in contrast to a shallow network, where the extracted features are more general.
- The output layer in a CNN is a fully connected layer. The input from the previous layers is flattened (vec) and transformed into a form that allows the network to generate an output whose dimension is equal to the number of classes we wish to recognize.
- The network generates the final output through the output layer and compares it with the expected result (the target class) in order to compute the error. In this layer, the loss function (e.g., mean squared error) is defined and used to compute the error gradient.
- The error is then propagated backward through the network (backpropagation) in order to update the filter weights and biases.
- One training cycle consists of one forward pass and one backward pass through the network (part of an epoch).

3 Forming the Algorithm

3.1 How to Compute the Optimal Coefficients θ

3.1.1 Gradient Descent

For a neural network f of depth $k \in \mathbb{N}$ parametrized by $u \in \mathbb{N}$ coefficients, with a total of m training samples, we solve the following problem:

$$\hat{f}^*(\hat{\theta}) = \min_{\theta \in \mathbb{R}^u} \hat{f}^*(\theta) = \min_{\theta \in \mathbb{R}^u} \frac{1}{m} \sum_{i=1}^m L(f_\theta(x_i), y_i)$$

where L is a specific loss function and $\hat{\theta}$ denotes the minimizer found. Under sufficient smoothness assumptions on the loss function, for example $L \in C^1$, we may use Taylor expansion to obtain the following approximation with quadratic error:

$$\hat{f}^*(\theta + \Delta\theta) \approx \hat{f}^*(\theta) + \sum_{i=1}^u \frac{\partial \hat{f}^*}{\partial x_i}(\theta) \cdot \Delta\theta_i$$

If we want to reach a minimizer, one strategy is to find $\bar{\theta}$ such that $(D\hat{f}^*)(\bar{\theta}) = 0$, by the **necessary condition for a minimizer**. The problem with this approach is that it finds a local minimum, but not necessarily a global one.

A **momentum** term is typically introduced to help the algorithm avoid such local minima; this will be discussed later.

The strategy is as follows: find a direction vector v together with a coefficient η (the so-called **learning rate**), and set the next iteration to be

$$\theta^* = \theta + \eta \cdot v$$

The question is how to determine the direction vector that will move us closest to the minimizer. From the **Cauchy-Schwarz** inequality, we obtain

$$\left| \sum_{i=1}^u \frac{\partial \hat{f}^*}{\partial x_i}(\theta) \cdot \Delta\theta_i \right| \leq \|\nabla \hat{f}^*(\theta)\| \cdot \|\Delta\theta\|$$

Equality is achieved if and only if there is linear dependence, that is, if we require

$$\Delta\theta = \frac{\nabla \hat{f}^*(\theta)}{\|\nabla \hat{f}^*(\theta)\|}$$

Thus, we expect the largest changes in this direction. Since we seek a minimum, we define

$$v := -\frac{\nabla \hat{J}^*(\theta)}{\|\nabla \hat{J}^*(\theta)\|}$$

although in practice the normalization step is not performed. In this way, we have derived the **gradient descent** algorithm.

3.1.2 Stochastic Gradient Descent

One of the problems with using the gradient descent algorithm is its high computational cost when the number of training samples is large. At each step, we must compute the gradient, and the sum for J^* contains m terms; thus, each step requires $m \cdot u$ summations.

Observe that the gradient of $\hat{J}^*$ has the form of an expectation:

$$\nabla \hat{J}^*(\theta) = \mathbb{E}_{x,y \sim \hat{p}_{\text{data}}} \nabla_{\theta} L(f_{\theta}(x), y)$$

This naturally suggests approximating the expectation using smaller groups of training samples, known as **mini-batches** (*minibatch*).

Namely, instead of taking the entire training set of length m , we take a subset of length c of the form $\{x_{i_1}, \dots, x_{i_c}\}$, most often chosen according to a **uniform distribution**, and compute

$$\nabla \hat{J}^*(\theta) \approx \frac{1}{c} \sum_{k=1}^c \nabla_{\theta} L(f_{\theta}(x_{i_k}), y_{i_k})$$

The advantage of this approach is that c is a parameter independent of the number of training samples, and after sufficiently many steps we expect the approximation error to disappear.

Nevertheless, there is deeper theory that provides conditions on the learning rate η under which the approximation is appropriate.

Ordinary gradient descent can tolerate a fixed value of the learning rate. In the stochastic approach, if ε_k denotes the learning rate at the k -th step, then the following conditions must hold:

$$\sum_{k=1}^{\infty} \varepsilon_k = \infty$$

$$\sum_{k=1}^{\infty} \varepsilon_k^2 < \infty$$

In practice, since the algorithm is not run for infinitely many steps, **linear decay** of the learning rate is typically used. If $\tau \in \mathbb{N}$ denotes the final decay step and we choose small values $\varepsilon_0, \varepsilon_\tau > 0$, then we set

$$\varepsilon_k = (1 - \frac{k}{\tau})\varepsilon_0 + \varepsilon_\tau, k = 1, \dots, \tau$$

after which the remaining sequence is constant with value ε_τ . Choosing the values $\varepsilon_0, \varepsilon_\tau, \tau$ is a separate problem and usually depends on the nature of the data on which learning is performed. Some commonly used rules are:

- ε_τ is about 1% of the value of ε_0 .
- τ is chosen together with the mini-batch size c so that the entire dataset can be traversed at least one hundred times.

Thus, the problem reduces to choosing a good value for ε_0 .

3.1.3 Momentum

In earlier chapters, we mentioned the problem of local minima. The goal of the algorithm is to reach the global minimizer, but the idea behind the method guarantees only that a local minimizer will be reached.

The intuition for mitigating this problem comes from the physical problem of a ball moving

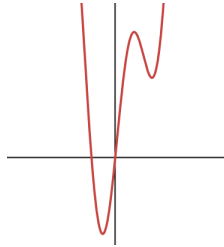


Figure 3.1: Global and local minimizer

along a curve. With a sufficient amount of motion, the ball will not stop abruptly in the region of a local minimizer, but will continue moving slightly around it. If it has a sufficiently large velocity, it may even leave that region and actually reach the global minimizer.

Formally, within the algorithm we introduce a variable v that imitates the “velocity” of the iteration point θ_k at the k -th step of the algorithm.

Assuming that this point has unit mass, the variable v describes its quantity of motion; this is

the origin of the term **momentum**.

A step of the modified stochastic gradient descent algorithm is given by the equations

$$v \leftarrow \alpha v - \eta \nabla_{\theta} \left(\frac{1}{c} \sum_{k=1}^c L(f_{\theta}(x_{i_k}), y_{i_k}) \right)$$

$$\theta \leftarrow \theta + v$$

where $\alpha \in [0, 1)$ is a hyperparameter that determines the decay of the influence of previous gradients.

In particular, the magnitude of the displacement toward the minimizer at the k -th step is considerably more difficult to determine because of the previous gradients. In general, if the direction of motion changes only slightly, we expect increasingly large velocities until a terminal velocity is reached, with value (assuming initial velocity $v_0 = 0$ and direction of motion g):

$$\frac{\eta \|g\|}{1 - \alpha}$$

Values from the set $\{0.5, 0.9, 0.99\}$ are often used for the hyperparameter α . It is also possible to change the values dynamically by increasing them over time, although this is less important than decreasing the learning rate η .

3.2 Computing Gradients

For easier computation (BGO), we consider the loss function $\mathcal{L}$ together with the neural network $f = f_k \circ f_{k-1} \circ \dots \circ f_1$, in the sense that $f_k = \mathcal{L}$. We will still treat f_k as if it had its own parameters θ_k , even though in reality they do not exist (so any number of parameters may be assigned; the gradient will always be 0).

The goal is to compute the derivative of the function f with respect to the parameters $\theta = \bigcup_{i=1}^k \theta_i$; that is, for the parameters $\theta_i, i = 1, \dots, k$:

$$D_{\theta_i} f(x : \theta) = D_{\theta_i} (f_k(\cdot : \theta_k) \circ \dots \circ f_1(\cdot : \theta_1))(x)$$

where the inner composition means

$$f_k(\cdot : \theta_k) \circ \dots \circ f_1(\cdot : \theta_1) := f_k(f_{k-1}(\dots(f_2(f_1(x_0 : \theta_1) : \theta_2) \dots) : \theta_k))$$

Using the more compact notation $x_i = f_i(x_{i-1} : \theta_i)$, where x_0 is the input data and x_k is the scalar output value, the chain rule gives

$$D_{\theta_i} f(x_0 : \theta) = D_{x_{k-1}} f_k(x_{k-1} : \theta_k) \cdot D_{x_{k-2}} f_{k-1}(x_{k-2} : \theta_{k-1}) \cdot \dots \cdot D_{x_i} f_{i+1}(x_i : \theta_{i+1}) \cdot D_{\theta_i} f_i(x_{i-1} : \theta_i)$$

Since f is a scalar function, $D_{\theta_i} f(x_0 : \theta)$ has the same dimensionality as θ_i . The main computational problem arises from carrying out the **inner multiplication**. The goal is to **construct a recursion** that avoids explicitly computing the inner Jacobian matrices.

Set $p_k = 1$. We then define the recursion

$$p_{l-1}^\top = p_l^\top \cdot D_{x_{l-1}} f_l(x_{l-1} : \theta_l), \quad l = i+1, \dots, k$$

In this way, by computing backward, we can obtain the desired derivative. The **main problems** are performing multiplication by the right Jacobian matrix and, in the last step, by the derivative of the function f_i with respect to the parameters θ_i .

Observe that

$$D_{x_l} (\langle p_{l+1}, f_{l+1}(x_l : \theta_{l+1}) \rangle) = p_{l+1}^\top \cdot D_{x_l} f_{l+1}(x_l : \theta_{l+1}) = p_l^\top$$

and analogously for the final computation:

$$D_{\theta_i} (\langle p_i, f_i(x_{i-1} : \theta_i) \rangle) = D_{\theta_i} f(x_0 : \theta)$$

We see that the computations of Jacobian matrices reduce to the computation of directional derivatives.

Overall, for each layer L in the neural network f , we want convenient ways to **compute a directional derivative** when the input x_L , the layer parameters θ_L , and the direction p are given.

3.3 Initializing the Recursion

Since the last layer will always be the evaluation of the loss function $\mathcal{L}$ and we begin with $p_k = 1$, we directly compute the Jacobian matrix with respect to the output data u_k . We first consider common loss functions used in classification problems; thus $u_{k-1} \in \mathbb{R}^C$, where the components are non-negative and sum to 1. The largest component is the model prediction.

3.3.1 Top K Loss Function

This function takes $x \in \mathbb{R}^C$ satisfying the conditions above as one argument, and $c \in 1, \dots, C$ as the other:

$$\mathcal{L}(x, c) = \mathbf{1}(|k : x_k \geq x_c| \leq K)$$

We see that for $K = 1$ we obtain the previously mentioned 0-1 loss function. This function is clearly not C^1 , but we still introduce it as motivation for the loss functions that follow.

3.3.2 Softmax

In cases where u_{k-1} is not guaranteed to be a column vector describing a discrete distribution, the following function may be added in composition with the loss function:

$$S : \{a_1, \dots, a_C\} \rightarrow [0, 1]$$

$$S(a_i) = \frac{e^{a_i}}{\sum_{l=1}^C e^{a_l}}$$

3.3.3 Log Loss Function

Also known as cross-entropy, this function is given by

$$\mathcal{L}_{\text{CCE}}(x, y) = - \sum_{j=1}^C y_j \log(x_j) = -\log(x_c)$$

where c is the index of the correctly classified component. One problem with this definition is that minimizing the expression does not penalize increased values of $x_t, t \neq c$; in practice, we want $x_c >> \sum_{i=1, i \neq c}^C x_i$.

3.3.4 Softmax Log Loss Function

Combining the two preceding functions gives

$$\mathcal{L}(x, c) = -\log\left(\frac{e^{x_c}}{\sum_{l=1}^C e^{x_l}}\right) = -x_c + \log\left(\sum_{l=1}^C e^{x_l}\right)$$

This avoids the problem of the ordinary log loss function. Furthermore, the second expression is used for numerical stability. Now, to differentiate with respect to the output data u_k , we

can compute the partial derivatives:

$$\frac{\partial \mathcal{L}}{\partial x_i}(x, c) = -\delta_{i,c} + \frac{e^{x_i}}{\sum_{l=1}^C e^{x_l}}$$

where δ is the Kronecker delta.

3.3.5 p-Norms, $1 \leq p < \infty$

In cases where we use loss functions defined through p-norms for $p \in [1, \infty)$, we have

$$\frac{\partial}{\partial x_i} \|x - y\|_p^p = p|x_i - y_i|^{p-1} \text{sign}(x_i - y_i)$$

Taking the p-th root gives a substantially more complicated formula:

$$\frac{\partial}{\partial x_k} \|x - y\|_p = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{\frac{1}{p}-1} \cdot |x_k - y_k|^{p-1} \text{sign}(x_k - y_k)$$

3.4 Carrying Out the Recursion – Activation Functions

Now that the recursion has been initialized, we need to show, for commonly used layers, how to obtain the projected derivative; equivalently, how to obtain the gradients of the k -th layer from the known gradients of the $(k+1)$ -st layer. This paragraph focuses specifically on activation layers.

3.4.1 ReLU

The ReLU function (*Rectified linear unit*) is given by

$$\text{ReLU} : \mathbb{R}^n \rightarrow \mathbb{R}^n, \quad [\text{ReLU}(x)]_i = \max\{0, x_i\}$$

This function is differentiable everywhere except at vectors containing a zero coordinate; in the one-dimensional case, denoted by ReLU , it is not differentiable at 0.

It is standard to define the derivative there as either the left or the right derivative (0 or 1), although in reality any value from $[0, 1]$ may be chosen, since this is the subgradient of ReLU at 0.

The theoretical justification lies in a **generalization** of the gradient descent method: the **sub-gradient method**, which is defined for convex functions without requiring differentiability.

For simplicity, we will continue using the term gradient descent even though, in reality, we rely on the subgradient method.

Thus, we set

$$(D_x \text{ReLU})(x) = \text{diag}(s)$$

where

$$s_i = \begin{cases} 0 & \text{if } x_i \leq 0 \\ 1 & \text{otherwise } x_i > 0 \end{cases}$$

for $i = 1, \dots, n$.

Hence, **in the recursion step**, for p_{l+1} and the output data x_l of the l -th layer, we return p_l as a vector that has 0 in the components where x_l is equal to 0, and is equal to p_{l+1} in the remaining components.

$$(p_l)_i = \begin{cases} 0, & \text{if } (x_l)_i \leq 0 \\ (p_{l+1})_i, & \text{otherwise } (x_l)_i > 0 \end{cases}$$

for $i = 1, \dots, n$.

More elegantly, using $\odot$ to denote the Hadamard product, we have

$$p_l = p_{l+1} \odot \mathbf{1}_{\{x_l > 0\}}$$

where the vector on the right is the indicator function of the positive components of the vector x_l .

3.4.2 Sigmoid – Logistic Function

The sigmoid σ is defined as

$$\begin{aligned} \sigma &: \mathbb{R} \rightarrow \mathbb{R} \\ \sigma(x) &= \frac{1}{1 + e^{-x}} \end{aligned}$$

We extend the action of the function to $\mathbb{R}^n$ componentwise; let this mapping be denoted by $\mathcal{O}$.

We have

$$\begin{aligned}
\frac{d\sigma}{dx}(x) &= \frac{-(-e^{-x})}{(1+e^{-x})^2} \\
&= \frac{e^{-x}}{(1+e^{-x})^2} \\
&= \sigma(x) - (\sigma(x))^2 \\
&= \sigma(x) \cdot (1 - \sigma(x))
\end{aligned}$$

Overall, we obtain

$$(D_x \mathcal{O})(x) = \text{diag}(t)$$

where

$$t_i = \sigma(x_i) \cdot (1 - \sigma(x_i))$$

for $i = 1, \dots, n$.

Hence, **in the recursion step**, for p_{l+1} and the output data x_l of the l -th layer, we return p_l as the vector with components

$$(p_l)_i = (p_{l+1})_i \cdot \sigma((x_l)_i) \cdot (1 - \sigma((x_l)_i))$$

for $i = 1, \dots, n$.

The compact notation is

$$p_l = p_{l+1} \odot \mathcal{O}(x_l) \odot (e - \mathcal{O}(x_l))$$

where e is the column vector of ones.

3.5 Carrying Out the Recursion – Pooling Layers

In this paragraph, we discuss pooling layers, whose main purpose is to reduce the dimensionality of the input data. They are defined in a way that takes into account the matrix or third-order tensor structure of the input data, which is how images are stored.

Let the input data be $x \in \mathbb{T}_{V,S,D}(\mathbb{R})$ (for $D = 1$, x is a matrix). For $V', S' \in \mathbb{N}$ and fixed $x_{i,j,k}$, consider the following subtensor:

$$M_{(i,j,k)} := x(i : (i + V' - 1), j : (j + S' - 1), k)$$

where $i \in \{1, \dots, V - V' + 1\}$, $j \in \{1, \dots, S - S' + 1\}$, and $k \in \{1, \dots, D\}$. Of course, this imposes conditions on the choice of variables V', S' : $1 \leq V' \leq V$ and $1 \leq S' \leq S$.

The **stride** parameter s further modifies the choice of the variables i, j as follows:

$$i \in \{1, \dots, V - V' + 1\} \cap \{t \in \mathbb{N} : t \equiv 1 \pmod{s}\}$$

$$j \in \{1, \dots, S - S' + 1\} \cap \{t \in \mathbb{N} : t \equiv 1 \pmod{s}\}$$

The **padding** parameter p further enlarges the input data at the boundaries by inserting trivial values, depending on the type of pooling being performed. This extension takes place in the first two dimensions.

p may be a single number (the same amount is added on all sides), an ordered pair (the first number determines the top-bottom extension and the second the left-right extension), or an ordered quadruple (each side has its own extension size).

For example, for $p = (p_1, p_2, p_3, p_4)$, on the layer $x(:, :, k) \in \mathbb{M}_{V,S}(\mathbb{R})$, $k \in \{1, \dots, D\}$ we would have $\hat{x}(:, :, k) \in \mathbb{M}_{V+p_1+p_3, S+p_2+p_4}(\mathbb{R})$, where

$$i \in \{1 - p_1, \dots, V + p_3\}$$

$$j \in \{1 - p_2, \dots, S + p_4\}$$

Combining all of the above, the variables i and j would have a total of V'' and S'' candidates, respectively. We do not compute them explicitly as functions of s, p, V, S, V' and S' .

3.5.1 Max Pooling

The mapping $g : \mathbb{T}_{V,S,D}(\mathbb{R}) \rightarrow \mathbb{T}_{V'',S'',D}(\mathbb{R})$ is given by

$$(g(x))_{i'',j'',k} = \max M_{(i'',j'',k)}$$

If the padding value p is nontrivial, then padding is performed with $-\infty$ or with some value that certainly cannot be maximal; the reason for this is explained below.

We again encounter the differentiability problem for the function g , which is resolved in exactly the same way as for the ReLU function.

We know that the function $\max$ on $\mathbb{R}^n$ is convex, and we state the following facts about it

without proof:

- It is differentiable on those parts of the domain where the maximum is attained **at exactly one coordinate**; the differential is the **canonical vector corresponding to that coordinate**.
- On the parts of the domain where the maximum is attained at the coordinates $(i_1, j_1, k), \dots, (i_K, j_K, k)$, the subdifferential is given by

$$\partial \max(x) = \text{conv}\{e_{(i_1, j_1, k)}, \dots, e_{(i_K, j_K, k)}\}$$

where the e 's are canonical vectors with a one at the indicated coordinate. In order to obtain as many zeros as possible in the gradient, one typically chooses **some canonical vector from the subdifferential**.

This motivates the padding procedure in the case of nontrivial p : we want to construct a suitable gradient that is nonzero; otherwise the learning procedure, i.e., gradient descent, would be ineffective.

Remark: The arguments of layer functions must be in vectorized form. Thus, although we work with x as a third-order tensor or matrix because this makes the pooling mappings easier to understand, in reality we work with $\text{Vec}(x)$. The same applies to the function value: the space $\mathbb{T}_{A,B,C}(\mathbb{R})$ is replaced by $\mathbb{R}^{A \cdot B \cdot C}$ for $A, B, C \in \mathbb{N}$.

Accordingly, we can arrange the elements of x in lexicographic order:

$$(\text{Vec}(x))^\top = [x_{1,1,1}, x_{2,1,1}, \dots, x_{V,1,1}, x_{1,2,1}, x_{2,2,1}, \dots, x_{V,S,D}]$$

and define canonical vectors of the form $e_{(i,j,k)}$ analogously.

For $1 \leq i'' \leq V''$, $1 \leq j'' \leq S''$, and $1 \leq k \leq D$, we have

$$(D_{\text{Vec}(x)^\top g_{(i'', j'', k)}})(\text{Vec}(x)^\top) = e_t^\top$$

where t is the coordinate of the maximal element of the matrix $M_{(i'', j'', k)}$ with respect to the input third-order tensor or matrix x . If there are multiple possible maximizers, we **choose the first** in lexicographic order.

This gives the full differential of the function g ; all that remains is to apply it to the recursion. Since it is necessary to know the indices at which the maximal elements of the relevant matri-

ces are attained, in practice these data are **stored** during the **ordinary forward application of the pooling layer**, rather than recomputed while carrying out the recursion. Thus, given p_{l+1} and the assignment $(i'', j'', k) \xrightarrow{\theta} t$ (the index of the maximal element of the matrix $M_{(i'', j'', k)}$), we have:

- Define p_l as the zero vector of length $S \cdot V \cdot D$.
- For $u = (1, 1, 1), \dots, (V'', S'', D)$, perform $p_l(\theta(u)) = p_l(\theta(u)) + p_{l+1}(u)$.

4 Experimental Results

In this chapter, we test three different CNN architectures. We begin with the simplest structure and then gradually extend it with additional layers and regularization mechanisms in order to improve the accuracy and robustness of the model. Each model is used for binary classification of a given news article: it determines whether the article should be labeled as true (REAL) or false (FAKE).

An important implementation detail is that each word is assigned a *token* (a natural number). If a word is assigned $n + 1 \in \mathbb{N}$, then it is the n -th most frequent word; if it does not appear among the *MAX_VOCAB_SIZE* most frequent words, it is assigned the value 1.

4.1 Model 1 – Basic CNN Architecture

The first model has a standard, simple architecture with the following layers:

1. **Embedding (dimension = 100)** Each token (word index) is assigned a 100-dimensional vector from a weight matrix of size (*MAX_VOCAB_SIZE*, 100). The weights are updated during training (*trainable=True*), so the embedding learns semantic relationships among words. The idea is that synonyms and similar words will end up in the same clusters in the large 100-dimensional space.
2. **SpatialDropout1D (rate = 0.2)** Randomly drops 20 % of entire embedding dimensions in order to prevent the model from depending too strongly on individual features and to encourage robustness. We do not want the model to rely on only a few coordinates and make accurate conclusions from them, because this is an ideal setting for *overfitting*. The aim is to involve all coordinates of the vector in the final decision.
3. **Conv1D (128 filters, kernel size = 5, ReLU)** The convolutional layer learns 128 filters that “slide” through the text and detect patterns consisting of exactly 5 consecutive words (5-grams). The ReLU activation introduces nonlinearity and sets all negative outputs to zero, thereby filtering weaker or less relevant matches. Intuitively, each filter learns to recognize a specific sequence of five words whose occurrence in the text is interpreted as a strong “signal” for the model’s final decision.
4. **GlobalMaxPooling1D** Compresses the spatial activations of all 128 neural maps by selecting, for each map, the largest value along the entire sequence. In this way, we ob-

tain a 128-dimensional vector in which each component represents the strongest “signal” of an individual neuron, i.e., its maximum level of match with the corresponding five-word pattern in the text.

5. **Dense (64 neurons, ReLU)** A fully connected layer with 64 neurons, each connected to all 128 input features from the previous layer. The ReLU activation introduces non-linearity and enables the model to learn complex, multi-level combinations of these features before final classification.
6. **Dropout (rate = 0.5)** Randomly drops 50 % of neurons in the Dense layer for additional regularization and reduced overfitting. The intuition is similar to that of the SpatialDropout1D layer: here, we do not want excessive dependence on a small number of neurons, since that would mean relying only on their strong signals.
7. **Dense (1 neuron, sigmoid)** The final model decision, with sigmoid activation, returns the probability of the REAL class (a value in the interval $[0, 1]$). A threshold of 0.5 is used for the final binary decision (REAL vs. FAKE).

The model was trained for 10 epochs with `batch_size=64`, using `binary_crossentropy` and the Adam optimizer (`learning_rate=0.001`). During training, we monitored both loss and accuracy on the validation set:

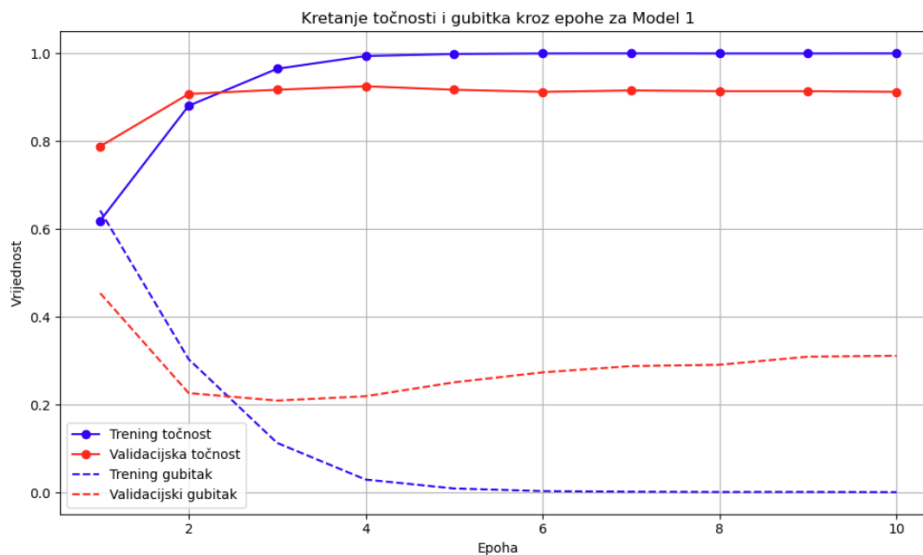


Figure 4.1: Accuracy and loss over epochs for Model 1.

After training, we evaluated the model on an unseen test set:

Test accuracy (Model 1) = 0.926

The confusion matrix for Model 1 shows the following values:

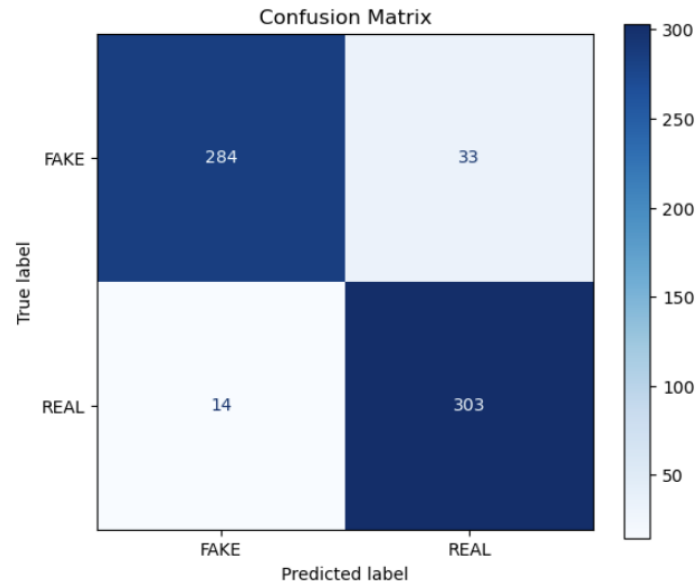


Figure 4.2: Confusion matrix for Model 1 on the test set.

From the matrix (Figure 4.2), we see:

- 284 correct FAKE classifications (*True Negatives*),
- 33 REAL articles incorrectly labeled as FAKE (*False Negatives*),
- 14 FAKE articles incorrectly labeled as REAL (*False Positives*),
- 303 correct REAL classifications (*True Positives*).

The overall robustness is good, although we observe that the model is somewhat weaker at detecting false news as false (33 *FN* cases).

Possible Improvements

- **Increasing network depth:** Adding an additional Conv1D or Dense layer may enable the model to learn more complex patterns.

- **Different kernel widths:** Introducing parallel convolutions with `kernel_size=3,5,7` helps detect phrases of different lengths.
- **Regularization and dropout rate:** Fine-tuning dropout rates or introducing an L2 regularizer may reduce *overfitting*; the graph shows particularly clearly that this is a major issue.
- **More advanced optimizers and learning-rate schedulers:** Introducing `ReduceLROnPlateau` and `EarlyStopping` may accelerate convergence and avoid unnecessary epochs, with emphasis on stabilizing accuracy already after 5 epochs.

4.2 Model 2 – Extended CNN Architecture with Additional Layers

The second model was extended with parallel convolutional branches and additional densely connected layers in order to improve the model's ability to capture diverse n-gram patterns and reduce overfitting. As a result, we made the following changes:

- We increased the embedding space dimension to 150 in order to obtain a richer semantic representation.
- We introduced `BatchNormalization` after each convolution to stabilize the distribution of activations.
- We added one more Dense layer (64 neurons) between the concatenation and the output to increase the model's capacity to learn abstract combinations of features.
- We kept Dropout at 0.5 for regularization.
- We used `EarlyStopping` and `ReduceLROnPlateau` to automatically stop training and adjust the learning rate based on the `val_loss` metric.

Figure 4.3 shows the evolution of accuracy and loss over epochs:

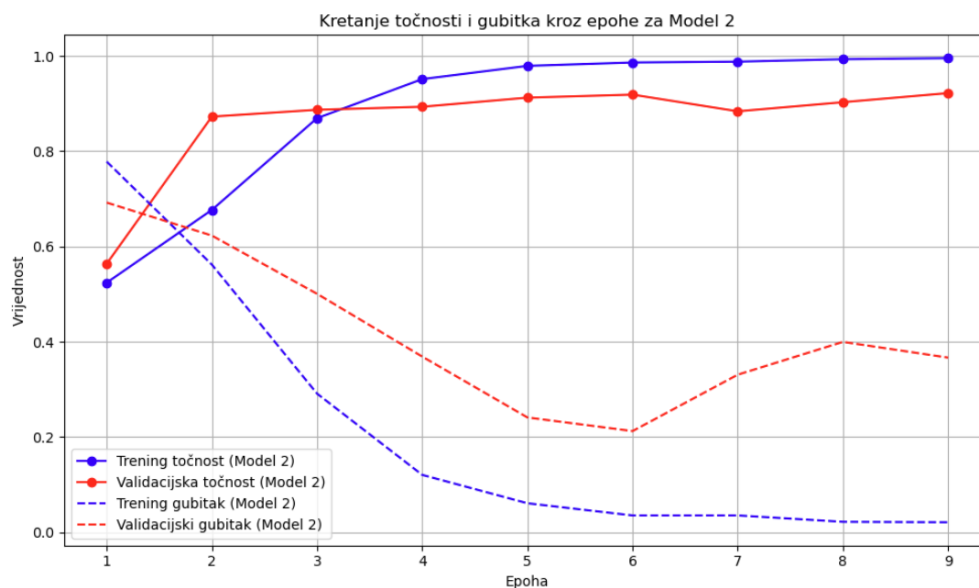


Figure 4.3: Training and validation accuracy, and training and validation loss, over epochs for Model 2.

From the graph, we can observe:

- **Rapid growth in training accuracy:** after only the 3rd epoch, training accuracy exceeds 90 %.
- **Best val_accuracy in epoch 6:** validation accuracy reaches a peak of about 92 %, but then decreases slightly, which may indicate the beginning of mild overfitting.
- **Minimum val_loss in epoch 5:** the lowest validation loss (about 0.21) also occurs in the middle of training, after which it increases; this is again a classical symptom of *overfitting*.
- **Stabilization:** thanks to ReduceLROnPlateau, the learning rate decreases after the plateau phase, allowing the model to maintain fairly stable performance until the end.

The final test accuracy for Model 2 was:

$$\text{Test accuracy}_2 \approx 0.9243$$

which is slightly below the performance of the first model on the test set, but with greater stability and better regularization mechanisms.

4.3 Recommendations for Model 3

For the next iteration (Model 3), we propose:

- Adding another convolutional level or introducing residual connections (Residual Blocks) for deeper feature extraction without additional *overfitting*.
- Experimenting with different dropout rates and L2 regularizers to further reduce the generalization gap between training and validation loss.
- Increasing the number of epochs with a more aggressive EarlyStopping criterion, since stabilizing training accuracy remains an unresolved issue.

4.4 Model 3 – Optimized CNN with BatchNorm and Stronger Regularization

The third model was created as a direct implementation of the recommendations from the previous experiments:

- **Higher dropout (rate = 0.6)** on the input embedding layer and the main Dense layer for more aggressive regularization.
- **L2 regularization** in all Conv1D and Dense layers with coefficient $\lambda = 10^{-3}$ in order to prevent the weights from being overlearned.
- **BatchNormalization** after each Conv1D layer and Dense layer for more stable and faster convergence.
- **Reduced number of filters** (32 instead of 64) and neurons (64) in order to simplify the model and reduce its capacity to memorize noise.
- **Parallel Conv1D paths** (3-, 5-, and 7-gram) were retained for extracting diverse local patterns.
- **EarlyStopping (patience=1)** and **ReduceLROnPlateau (patience=1)** were used so that training would stop as soon as *val_loss* stabilized and the final best weights would be obtained.

As in the previous models, we monitor accuracy and loss during each step:

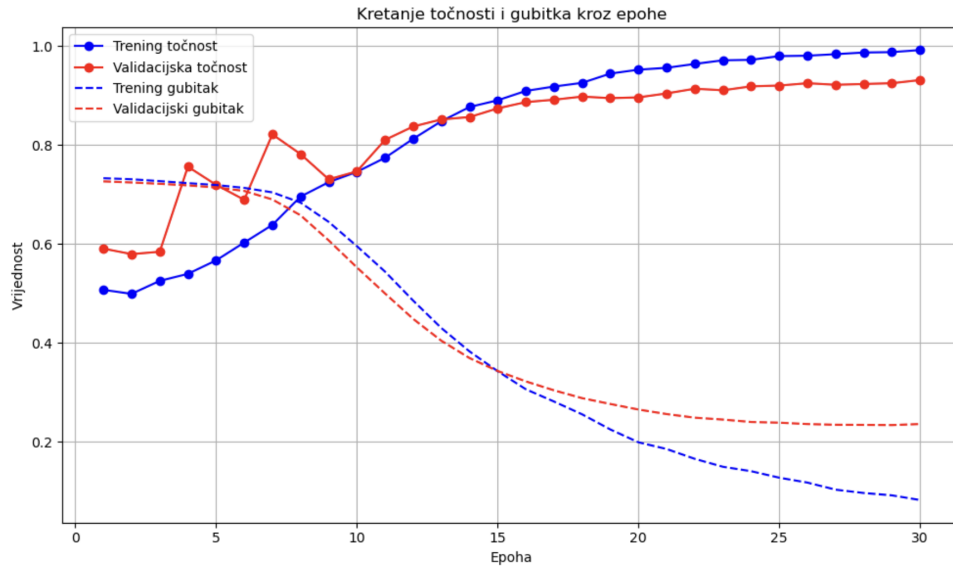


Figure 4.4: Training and validation accuracy, and training and validation loss, over epochs for Model 3.

From the graph (Figure 4.4), we can see:

- Training and validation quickly align around 0.90–0.95, without the large gap observed in the previous versions; the *overfitting* problem is no longer present.
- Validation loss remains stable (around 0.24) and continues to decrease slightly, confirming that the model is still learning features but is not beginning to “memorize” noise.
- EarlyStopping was activated after only a few epochs, which means that the optimizer quickly found an optimal region and then stopped training once further epochs no longer brought improvement.

Finally, we tested the model on an unseen set:

Test accuracy (Model 3) ≈ 0.94

The confusion matrix for Model 3 is as follows:

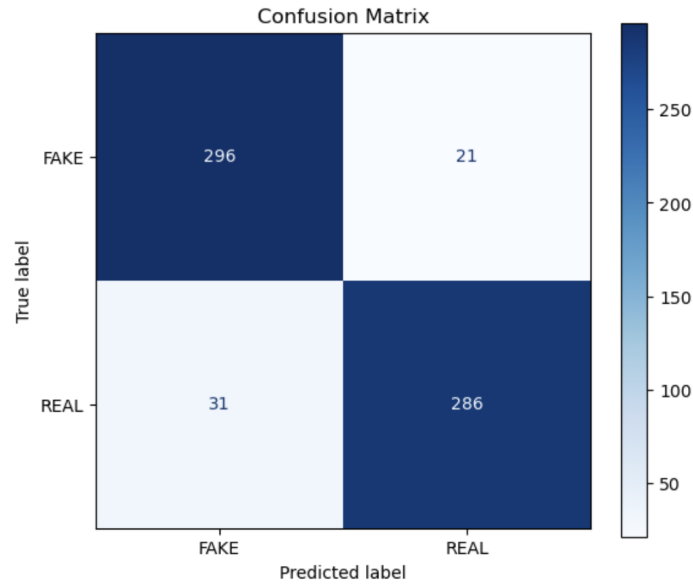


Figure 4.5: Confusion matrix for Model 3 on the test set.

4.5 Conclusion of the Experimental Part

A comparison of the results of all three models clearly shows:

1. **Model 1** achieved good baseline accuracy (≈ 0.93), but had a substantial gap between training and validation.
2. **Model 2** introduced batch normalization and additional layers, which stabilized the learning process, but it still showed slight overfitting after 6 epochs.
3. **Model 3**, by intelligently combining stronger regularization (dropout + L2), normalization (BatchNorm), and reduced complexity (fewer filters and neurons), achieved the most robust behavior and the highest test accuracy (≈ 0.94).

This progress illustrates how iterative fine-tuning of architecture and hyperparameters can significantly improve the ability of convolutional networks to generalize to authentic and fake news.

5 Conclusion

This seminar examined one of the most powerful and widely used structures in deep learning: the convolutional neural network (CNN). Beginning with the fundamental mathematical concepts of machine learning, we defined basic notions such as the loss function, empirical risk optimization, the role of training and test sets, and challenges such as overfitting and the choice of an appropriate function family. We also analyzed in detail the mathematical optimization techniques underlying the training of neural networks, including gradient descent, stochastic gradient descent, and momentum, and showed how gradients are computed using the recursive structure of the network.

After establishing these foundations, we focused on convolutional neural networks, which introduce the ideas of local connectivity and weight sharing, enabling efficient scaling to high-dimensional data such as images and text. Through practical examples, we presented CNN architectures with convolutional, activation, and pooling layers, emphasizing their role in feature extraction, feature aggregation, and pattern-based classification.

The practical part of the seminar addressed the problem of classifying news as true or false. We implemented two different CNN models in Python using Keras and TensorFlow and trained them on a real dataset. The first model served as a baseline, while the second was improved through additional convolutional layers, batch normalization, and a higher density of neural connections. The experiments, including graphical representations of training and validation metrics, a confusion matrix, and the classification of real and fabricated articles (including an April Fools' joke from the portal Index.hr), demonstrated an exceptionally high accuracy of 93.8% for the second model.

Thus, we covered the key aspects of working with convolutional networks both theoretically and practically: from architectural construction and optimization to performance evaluation. CNNs proved robust even in problem domains that are not image-based, such as text analysis and misinformation detection.

This work emphasizes the importance of a solid theoretical foundation for understanding neural networks, as well as the need for experimental validation of their effectiveness. We showed that by connecting theory and practice, we can develop models that are not only accurate but also adaptable to real challenges in modern computing. In the future, this work could be extended by using pretrained language representations such as BERT, introducing multi-class classification, and applying explainable artificial intelligence (XAI) methods in order to better understand the model's decisions.

6 References

- Catherine F. Higham, Desmond J. Higham; *Deep Learning: An Introduction for Applied Mathematicians*, SIAM REVIEW Vol. 61, No. 4, pp. 860–891
- Andrea Vedaldi, Karel Lenc, Ankush Gupta; *Convolutional Neural Networks for MATLAB*
- Daniel McCarter ; *MATHEMATICAL ANALYSIS OF CONVOLUTIONAL NEURAL NETWORKS*
- Ian Goodfellow, Yoshua Bengio, Aaron Courville; *Deep Learning*